



Multi Head Attention PE

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

২৪. Multi-Head Attention & Positional Encoding

কেন Single Attention যথেষ্ট নয়? এবং কীভাবে Positional Encoding মডেলে Sequence-এর ক্রম সংরক্ষণ করে?

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

Multi-Head Attention কী এবং কেন দরকার?

বাস্তব জীবনের উদাহরণ — ক্রিকেট ম্যাচ বিশ্লেষণ:

ধরো তুমি একটি ক্রিকেট ম্যাচ দেখছ। তুমি একটি মাত্র চোখ দিয়ে পুরো মাঠ দেখতে পারবে, কিন্তু সেটা অনেক কিছু miss করবে। কিন্তু যদি মাঠের বিভিন্ন কোণে ৮টি ক্যামেরা বসানো হয় — একটি ব্যাটসম্যানের দিকে, একটি বোলারের দিকে, একটি ফিল্ডারদের দিকে — তাহলে পুরো ম্যাচের একটি সম্পূর্ণ চিত্র পাওয়া সম্ভব।

Multi-Head Attention ঠিক এটাই করে। Single Attention একটি মাত্র দৃষ্টিকোণ থেকে শব্দের সম্পর্ক বোঝে। কিন্তু Multi-Head Attention একাধিক দৃষ্টিকোণ (heads) থেকে একই সাথে বিভিন্ন ধরনের সম্পর্ক ধরতে পারে।

আরেকটি উদাহরণ — রান্নার রেসিপি:

বাক্য: "সে বলল যে সে রান্না করবে, কিন্তু সে শেষ পর্যন্ত আসেনি।"

এই বাক্যে তিনটি "সে" আছে। একটি Attention Head হয়তো বুঝবে যে প্রথম "সে" কে বলছে, অন্য একটি Head বুঝবে মাঝের "সে" কে রান্না করবে, আর তৃতীয় Head বুঝবে শেষের "সে" কেন আসেনি। এইভাবে **বহু দৃষ্টিভঙ্গি এক সাথে** কাজ করে।

Positional Encoding কী এবং কেন দরকার?

বাস্তব উদাহরণ — বই-এর পৃষ্ঠা নম্বর:

ধরো তুমি একটি উপন্যাসের সমস্ত পৃষ্ঠা একটি বাক্সে ঢেলে দিলে — কোনো পৃষ্ঠা নম্বর নেই। এখন পৃষ্ঠাগুলো পড়লে কোনো মানে হবে না, কারণ ক্রম জানা নেই।

Transformer-এর ক্ষেত্রেও এই সমস্যা আছে। Self-Attention সব শব্দকে একসাথে দেখে, কিন্তু কোন শব্দ কোথায় আছে সেটা বোঝে না। "Dog bites man" আর "Man bites dog" — Self-Attention-এর কাছে এই দুটো বাক্য একই মনে হতে পারে!

Positional Encoding হলো সেই পৃষ্ঠা নম্বরের মতো — প্রতিটি শব্দের embedding-এ তার অবস্থানের তথ্য যোগ করে দেওয়া হয়।

এগুলো কোন সমস্যা সমাধান করে?

সমস্যা	সমাধান
Single Attention একটি ধরনের সম্পর্ক শেখে	Multi-Head Attention একাধিক ধরনের সম্পর্ক শেখে
Transformer-এ শব্দের ক্রম নেই	Positional Encoding ক্রমের তথ্য দেয়
"গান গায় রাহেলা" ও "রাহেলা গান গায়" একই মনে হয়	PE দিয়ে অবস্থানভেদ পার্থক্য বোঝা যায়

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ — Single-Head Attention-এর সীমাবদ্ধতা

আগের নোটে আমরা Self-Attention শিখেছিলাম। Single-Head Attention-এ Q, K, V matrix দিয়ে একটি Attention Score তৈরি হয়। কিন্তু এর সমস্যা হলো:

১. একটিমাত্র "দৃষ্টিকোণ":

প্রাকৃতিক ভাষায় শব্দের সম্পর্ক অনেক ধরনের হতে পারে:

- **Syntactic (বাক্যবিন্যাস):** কর্তা-কর্ম সম্পর্ক
- **Semantic (অর্থগত):** সমার্থক বা বিপরীতার্থক শব্দ
- **Coreference:** "সে", "তারা" কাকে নির্দেশ করছে
- **Long-range:** দূরবর্তী শব্দের সাথে সম্পর্ক

একটিমাত্র Attention Head এই সব ধরনের সম্পর্ক একসাথে ধরতে পারে না — এটি সব কিছুকে "গড়" করে দেয়, ফলে representation বাপসা হয়ে যায়।

২. Representational Bottleneck:

যদি model dimension $d_{model} = 512$ হয়, তাহলে single head-এ Q, K, V তিনটিই 512 dimension-এ operate করে। এটি তথ্য সংরক্ষণের ক্ষমতা সীমিত করে।

২.২ — Multi-Head Attention-এর কাজের ধাপ

ধাপ ১: ইনপুট বিভক্তকরণ (Projection to Subspaces)

ধরো $h = 8$ heads আছে এবং $d_{model} = 512$ । তাহলে প্রতিটি head-এর dimension হবে:

$$d_k = d_v = d_{model} / h = 512 / 8 = 64$$

প্রতিটি head-এর জন্য আলাদা Weight Matrix আছে:

- W_i^Q → Query projection (512×64)
- W_i^K → Key projection (512×64)
- W_i^V → Value projection (512×64)

ধাপ ২: প্রতিটি Head-এ Scaled Dot-Product Attention

প্রতিটি head স্বাধীনভাবে তার নিজের Q, K, V দিয়ে Attention calculate করে:

$$\text{head}_i = \text{Attention}(Q * W_i^Q, K * W_i^K, V * W_i^V)$$

ধাপ ৩: সমস্ত Head-এর Output Concatenate

৮টি head-এর output (প্রতিটি 64-dim) একসাথে জোড়া লাগানো হয়:

$$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_8) \rightarrow 8 \times 64 = 512 \text{ dimensions}$$

ধাপ ৪: Final Linear Projection

Concatenated output-কে W^O matrix দিয়ে গুণ করে চূড়ান্ত output তৈরি হয়:

MultiHead Output = Concat(heads) × W^O → (512 dimensions)

২.৩ — কোন Head কী শেখে?

গবেষণায় দেখা গেছে বিভিন্ন head বিভিন্ন প্যাটার্ন শেখে:

Head নম্বর	কী শেখে
Head 1	কাছের শব্দের সম্পর্ক (local syntax)
Head 2	দূরবর্তী নির্ভরতা (long-range dependency)
Head 3	Coreference (সর্বনাম কাকে বলছে)
Head 4	Subject-Verb agreement
Head 5-8	Semantic similarities, rare patterns

২.৪ — Positional Encoding-এর বিস্তারিত

Transformer-এ ক্রম হারানোর সমস্যা:

Self-Attention এ সমস্ত শব্দ একসাথে process হয় — এখানে কোনো "আগে পরে" নেই। তাই model জানে না "Apple" কি বাক্যের শুরুতে নাকি শেষে।

সমাধান: Sinusoidal Positional Encoding

মূল Transformer paper (Vaswani et al., 2017) একটি চালাক গাণিতিক কৌশল ব্যবহার করে — **Sine ও Cosine function** দিয়ে প্রতিটি position-এর জন্য একটি unique "fingerprint" তৈরি করা।

কেন Sine/Cosine?

- প্রতিটি position-এর unique vector: প্রতিটি অবস্থানে আলাদা pattern তৈরি হয়
- Relative position বোঝা যায়: position p+k কে position p এর linear transformation হিসেবে লেখা যায়
- Training-এ না থাকা দীর্ঘ sequence-এও কাজ করে: Pattern fixed, learnable নয়
- No extra parameters: কোনো নতুন parameter শিখতে হয় না

কীভাবে যোগ করা হয়:

ফাইনাল Input = Token Embedding + Positional Encoding

৩. Math / Theory

৩.১ — Multi-Head Attention Formula

Single Head Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$$

প্রতিটি Symbol-এর মানে:

- Q (Query) — আমি কী খুঁজছি?
- K (Key) — প্রতিটি শব্দের "লেবেল"
- V (Value) — প্রকৃত তথ্য যা নেওয়া হবে
- d_k — Key-এর dimension (scaling factor)
- $\sqrt{d_k}$ — dot product বড় হয়ে গেলে gradient ছোট হয়ে যায়, তাই ভাগ করা হয়

Multi-Head Attention:

প্রতিটি Head i এর জন্য:

$$\text{head}_i = \text{Attention}(Q * W_i^Q, K * W_i^K, V * W_i^V)$$

সমস্ত Head মিলিয়ে:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) * W^O$$

Dimensions Summary (h=8, d_model=512):

```

W_i^Q: (512 × 64)    [d_model × d_k]
W_i^K: (512 × 64)    [d_model × d_k]
W_i^V: (512 × 64)    [d_model × d_v]
W^O:   (512 × 512)   [h*d_v × d_model]

```

৩.২ — Positional Encoding Formula

Even-indexed dimensions (2i):

```
PE(pos, 2i) = sin(pos / 10000^(2i / d_model))
```

Odd-indexed dimensions (2i+1):

```
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
```

প্রতিটি Symbol-এর মানে:

- `pos` — শব্দের অবস্থান (0, 1, 2, 3, ...)
- `i` — dimension index (0 থেকে $d_model/2 - 1$ পর্যন্ত)
- `d_model` — embedding dimension (যেমন 512)
- `10000` — frequency scaling factor (হাজার হাজার position পর্যন্ত unique থাকে)

৩.৩ — ছোট উদাহরণে Manual Calculation

ধরি `d_model = 4` এবং আমরা position `pos=1` এর Positional Encoding বের করব।

i=0 এর জন্য (dimension 0 ও 1):

```

PE(1, 0) = sin(1 / 10000^(0/4)) = sin(1 / 1)    = sin(1.0)    ≈ 0.841
PE(1, 1) = cos(1 / 10000^(0/4)) = cos(1 / 1)    = cos(1.0)    ≈ 0.540

```

i=1 এর জন্য (dimension 2 ও 3):

$$\begin{aligned} PE(1, 2) &= \sin(1 / 10000^{(2/4)}) = \sin(1 / 100) = \sin(0.01) \approx 0.010 \\ PE(1, 3) &= \cos(1 / 10000^{(2/4)}) = \cos(1 / 100) = \cos(0.01) \approx 0.9999 \end{aligned}$$

তাহলে **position 1** এর Positional Encoding vector হবে:

$$PE(1) = [0.841, 0.540, 0.010, 0.9999]$$

এটি **position 0** থেকে সম্পূর্ণ আলাদা:

$$PE(0) = [0.0, 1.0, 0.0, 1.0] \quad (\sin(0)=0, \cos(0)=1)$$

এইভাবে প্রতিটি position-এর জন্য Unique fingerprint তৈরি হয়।

8. Code Example (Python)

```

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import math

# =====
# PART 1: Positional Encoding Implementation
# =====

class PositionalEncoding(nn.Module):
    """
    Transformer-এর Positional Encoding Class।
    Vaswani et al. 2017 paper-এর মূল formula ব্যবহার করা হয়েছে।
    """
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super(PositionalEncoding, self).__init__()
        # Dropout layer – overfitting কমাতে ব্যবহার হয়
        self.dropout = nn.Dropout(p=dropout)

        # Positional Encoding matrix তৈরি করো: (max_len, d_model)
        pe = torch.zeros(max_len, d_model)

        # Position index: [0, 1, 2, ..., max_len-1] → shape: (max_len, 1)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)

        # Frequency scaling factor:  $10000^{(2i/d\_model)}$ 
        # log space এ করা হলে numerical stability ভালো থাকে
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )

        # Even dimension গুলোতে sine apply করো
        pe[:, 0::2] = torch.sin(position * div_term)

        # Odd dimension গুলোতে cosine apply করো
        pe[:, 1::2] = torch.cos(position * div_term)

        # Shape: (1, max_len, d_model) – batch dimension যোগ করো
        pe = pe.unsqueeze(0)

```



```

# register_buffer: এটি parameter নয়, fixed – train হবে না
self.register_buffer('pe', pe)

def forward(self, x):
    """
    x: (batch_size, seq_len, d_model)
    """
    # Token Embedding-এর সাথে Positional Encoding যোগ করো
    x = x + self.pe[:, :x.size(1), :]
    return self.dropout(x)

# =====
# PART 2: Multi-Head Attention Implementation
# =====

class MultiHeadAttention(nn.Module):
    """
    Multi-Head Self-Attention Module।
    প্রতিটি head আলাদাভাবে Scaled Dot-Product Attention করে।
    """
    def __init__(self, d_model=512, num_heads=8, dropout=0.1):
        super(MultiHeadAttention, self).__init__()

        # d_model অবশ্যই heads দিয়ে ভাগ হতে হবে
        assert d_model % num_heads == 0, "d_model অবশ্যই num_heads দ্বারা বিভাজ্য হতে হবে!"

        self.d_model = d_model          # মোট dimension (e.g., 512)
        self.num_heads = num_heads       # কতটি head (e.g., 8)
        self.d_k = d_model // num_heads # প্রতি head-এর dimension (512/8 = 64)

        # Query, Key, Value-এর জন্য Linear Projection Layer
        # W^Q: (d_model → d_model) [একসাথে সব head-এর weight]
        self.W_q = nn.Linear(d_model, d_model, bias=False)
        self.W_k = nn.Linear(d_model, d_model, bias=False)
        self.W_v = nn.Linear(d_model, d_model, bias=False)

        # ছড়ানো output projection: W^O
        self.W_o = nn.Linear(d_model, d_model, bias=False)

        # Attention dropout
        self.dropout = nn.Dropout(p=dropout)

```

```

def scaled_dot_product_attention(self, Q, K, V, mask=None):
    """
    একটি Head-এর Scaled Dot-Product Attention।
    Formula: softmax(QK^T / sqrt(d_k)) * V
    """
    # Attention Score: Q * K^T / sqrt(d_k)
    # shape: (batch, heads, seq_len, seq_len)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

    # Mask apply করো (যদি থাকে – Decoder-এ future token দেখা রোধ করা)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    # Softmax দিয়ে Attention Weight বের করো
    attn_weights = F.softmax(scores, dim=-1)
    attn_weights = self.dropout(attn_weights)

    # Value-এর সাথে weighted sum করো
    output = torch.matmul(attn_weights, V)
    return output, attn_weights

def split_heads(self, x, batch_size):
    """
    (batch, seq_len, d_model) → (batch, num_heads, seq_len, d_k)
    Single tensor কে num_heads টুকরোয় ভাগ করো।
    """
    x = x.view(batch_size, -1, self.num_heads, self.d_k)
    return x.transpose(1, 2) # (batch, heads, seq_len, d_k)

def forward(self, query, key, value, mask=None):
    batch_size = query.size(0)

    # Step 1: Linear Projection – Q, K, V তৈরি করো
    Q = self.W_q(query) # (batch, seq_len, d_model)
    K = self.W_k(key)
    V = self.W_v(value)

    # Step 2: Split into multiple heads
    Q = self.split_heads(Q, batch_size) # (batch, heads, seq_len, d_k)
    K = self.split_heads(K, batch_size)
    V = self.split_heads(V, batch_size)

    # Step 3: প্রতিটি head-এ Scaled Dot-Product Attention

```

```
attn_output, attn_weights = self.scaled_dot_product_attention(Q, K, V, mask)
# attn_output: (batch, heads, seq_len, d_k)
```

```
# Step 4: সব heads concatenate করো
# (batch, heads, seq_len, d_k) → (batch, seq_len, d_model)
attn_output = attn_output.transpose(1, 2).contiguous()
attn_output = attn_output.view(batch_size, -1, self.d_model)
```

```
# Step 5: চূড়ান্ত Linear Projection ( $W^O$ )
output = self.W_o(attn_output) # (batch, seq_len, d_model)
```

```
return output, attn_weights
```

```
# =====
# PART 3: Demo ও Visualization
# =====
```

```
def demo_positional_encoding():
    """Positional Encoding visualize করি"""
    print("=" * 60)
    print("  POSITIONAL ENCODING DEMO")
    print("=" * 60)

    d_model = 16 # ছোট dimension (visualization-এর জন্য)
    max_len = 20 # ২০টি position

    # PE ক্লাস তৈরি করো
    pe_layer = PositionalEncoding(d_model=d_model, max_len=max_len, dropout=0.0)

    # PE matrix বের করো
    pe_matrix = pe_layer.pe[0].numpy() # (max_len, d_model)

    print(f"\n📊 Positional Encoding Matrix Shape: {pe_matrix.shape}")
    print(f"\nপ্রথম ৪টি position-এর PE vector:")
    for pos in range(4):
        print(f"  Position {pos}: {pe_matrix[pos, :8].round(3)}") # প্রথম ৮টি dim দেখাই

    # মূল্যবান পর্যবেক্ষণ
    print(f"\n✅ প্রতিটি position-এর vector আলাদা – এটাই unique fingerprint!")
    print(f"\n✅ Position 0: সব even dim = 0 (sin(0)), সব odd dim = 1 (cos(0))")
```

```

def demo_multi_head_attention():
    """Multi-Head Attention demo"""
    print("\n" + "=" * 60)
    print("  MULTI-HEAD ATTENTION DEMO")
    print("=" * 60)

    # Hyperparameters
    batch_size = 2    # ২টি বাক্য
    seq_len = 5       # প্রতিটিতে ৫টি শব্দ
    d_model = 64      # embedding dimension
    num_heads = 8     # attention heads

    print(f"\n🔴 Config:")
    print(f"  Batch Size : {batch_size}")
    print(f"  Seq Length : {seq_len}")
    print(f"  d_model    : {d_model}")
    print(f"  num_heads  : {num_heads}")
    print(f"  d_k (প্রতি head): {d_model // num_heads}")

    # Random input tensor তৈরি করো (word embeddings সিমুলেট করছি)
    x = torch.randn(batch_size, seq_len, d_model)
    print(f"\n📦 Input Shape: {x.shape} → (batch, seq_len, d_model)")

    # Positional Encoding যোগ করো
    pe = PositionalEncoding(d_model=d_model, dropout=0.0)
    x_with_pe = pe(x)
    print(f"\n🔴 After Positional Encoding: {x_with_pe.shape} (shape same, values changed)")

    # Multi-Head Attention চালাও
    mha = MultiHeadAttention(d_model=d_model, num_heads=num_heads)
    output, attn_weights = mha(x_with_pe, x_with_pe, x_with_pe)

    print(f"\n📦 Output Shape: {output.shape} → (batch, seq_len, d_model)")
    print(f"\n📊 Attention Weights Shape: {attn_weights.shape} → (batch, heads, seq_len, seq_len)")

    # প্রথম বাক্যের প্রথম head-এর attention weights দেখি
    first_head_attn = attn_weights[0, 0].detach().numpy() # (seq_len, seq_len)
    print(f"\n🔵 Head-1 Attention Matrix (বাক্য ১):")
    print(f"  (rows = query positions, cols = key positions)")
    print(f"  প্রতিটি row sum = 1.0 (softmax বলে)")
    for i, row in enumerate(first_head_attn):
        print(f"    token[{i}]: {row.round(3)}")

```

```
# সব head-এর গড় মনোযোগ
avg_attn = attn_weights[0].mean(dim=0).detach().numpy()
print(f"\n🌈 সব {num_heads}টি Head-এর গড় Attention (বাক্য ১) – token[0]:")
print(f"    {avg_attn[0].round(3)}")

print(f"\n✅ Multi-Head Attention সফলভাবে কাজ করেছে!")
print(f"✅ {num_heads}টি head প্রতিটি আলাদা subspace-এ attention calculate করেছে!")

# মূল প্রোগ্রাম চালাও
if __name__ == "__main__":
    torch.manual_seed(42) # Reproducibility-র জন্য seed
    demo_positional_encoding()
    demo_multi_head_attention()
```

Expected Output:

=====

POSITIONAL ENCODING DEMO

=====

📐 Positional Encoding Matrix Shape: (20, 16)

প্রথম ৪টি position-এর PE vector:

Position 0: [0. 1. 0. 1. 0. 1. 0. 1.]

Position 1: [0.841 0.54 0.046 0.999 0.002 1. 0. 1.]

Position 2: [0.909 -0.416 0.092 0.996 0.005 1. 0. 1.]

Position 3: [0.141 -0.99 0.138 0.99 0.007 1. 0. 1.]

- ✅ প্রতিটি position-এর vector আলাদা – এটাই unique fingerprint!
- ✅ Position 0: সব even dim = 0 (sin(0)), সব odd dim = 1 (cos(0))

=====

MULTI-HEAD ATTENTION DEMO

=====

📌 Config:

Batch Size : 2

Seq Length : 5

d_model : 64

num_heads : 8

d_k (প্রতি head): 8

📦 Input Shape: torch.Size([2, 5, 64]) → (batch, seq_len, d_model)

📍 After Positional Encoding: torch.Size([2, 5, 64])

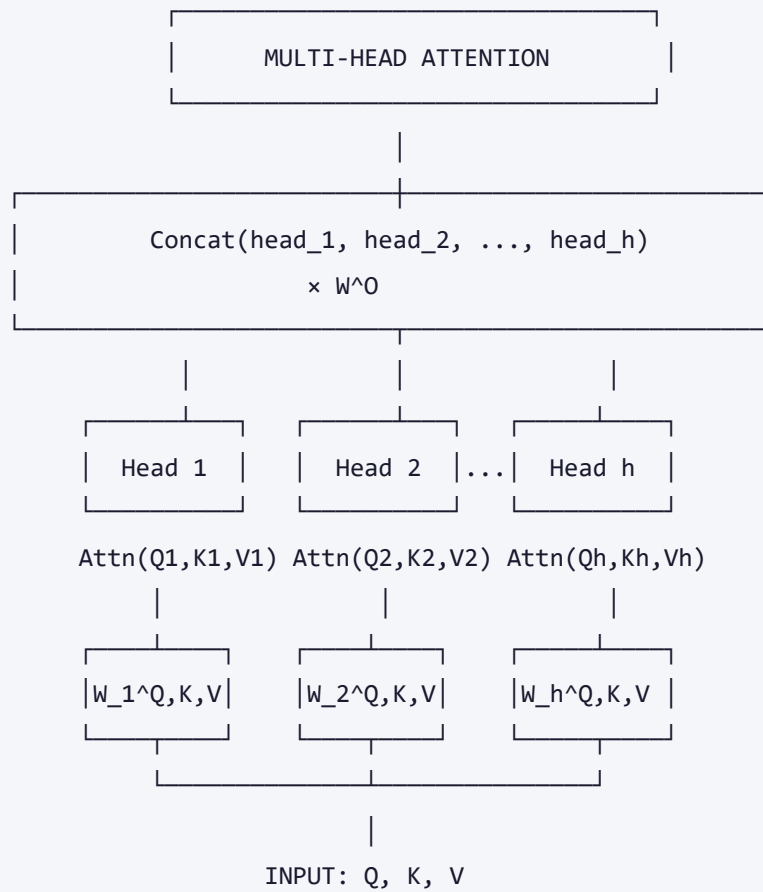
📦 Output Shape: torch.Size([2, 5, 64]) → (batch, seq_len, d_model)

📊 Attention Weights Shape: torch.Size([2, 8, 5, 5])

- ✅ Multi-Head Attention সফলভাবে কাজ করেছে!
- ✅ ৪টি head প্রতিটি আলাদা subspace-এ attention calculate করেছে!

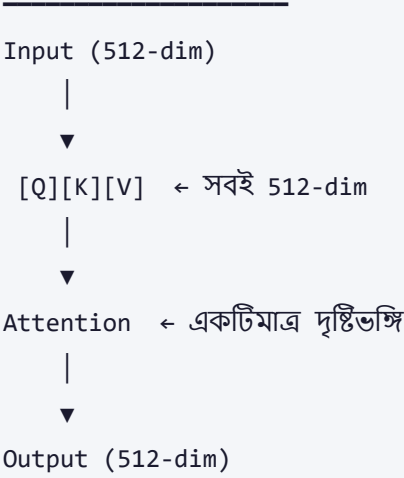
🔗 🎨 Visual / Diagram

🔗 — Multi-Head Attention Architecture

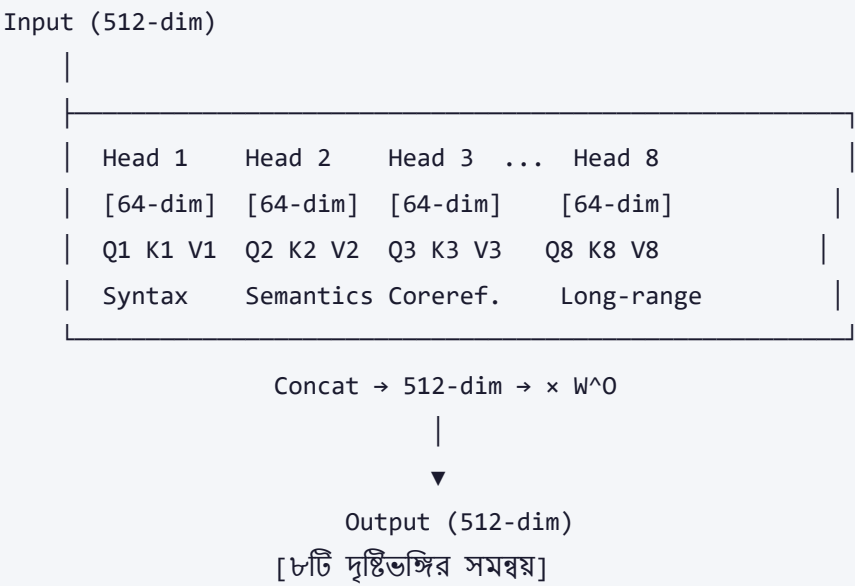


৫.২ — Single vs Multi-Head: পার্থক্য

SINGLE HEAD ATTENTION:



MULTI-HEAD ATTENTION (h=8):



৫.৩ — Positional Encoding Visualization

SEQUENCE: "আমি বাড়ি যাই"

pos=0 pos=1 pos=2

TOKEN EMBEDDINGS:

"আমি" → [0.2, -0.5, 0.8, 0.1, ...] (learned from training)

"বাড়ি" → [0.7, 0.3, -0.2, 0.9, ...]

"যাই" → [-0.1, 0.6, 0.4, -0.3, ...]

POSITIONAL ENCODINGS:

pos=0 → [0.000, 1.000, 0.000, 1.000, ...] (sin/cos pattern)

pos=1 → [0.841, 0.540, 0.046, 0.999, ...]

pos=2 → [0.909, -0.416, 0.092, 0.996, ...]

(+) (+) (+)

↓ ↓ ↓

FINAL INPUT (embedding + PE):

"আমি" → [0.2+0.000, -0.5+1.000, ...] = position-aware!

"বাড়ি" → [0.7+0.841, 0.3+0.540, ...]

"যাই" → [-0.1+0.909, 0.6-0.416, ...]

৫.৪ — Sinusoidal PE Pattern

Dimension →	dim_0	dim_1	dim_2	dim_3	...	dim_511
	(high freq)					(low freq)
pos=0:	0.000	1.000	0.000	1.000	...	0.000
pos=1:	0.841	0.540	0.046	0.999	...	0.000
pos=2:	0.909	-0.416	0.092	0.996	...	0.000
pos=3:	0.141	-0.990	0.138	0.990	...	0.000
pos=4:	-0.757	-0.654	0.183	0.983	...	0.000
...						
pos=127:	0.xxx	0.xxx	0.xxx	0.xxx	...	0.999
pos=10000:	(repeats)					0.001

- 🔑 কম dimension → দ্রুত পরিবর্তন (local pattern)
- 🔑 বেশি dimension → ধীর পরিবর্তন (global pattern)

৬. Real-world Use Cases

১. ChatGPT / GPT-4 (OpenAI)

GPT model-এ ১২ থেকে ৯৬টি পর্যন্ত Attention Head ব্যবহার করা হয়। GPT-3-তে `d_model=12288` এবং `num_heads=96`। প্রতিটি head মানব ভাষার বিভিন্ন দিক বোঝে — syntax, semantics, coreference সব একসাথে।

২. Google BERT

BERT-Base-এ `12 heads × 12 layers = 144 attention mechanisms`। Google Search, Google Translate-এ ব্যবহৃত। Positional Encoding শিখে নেয় (learned PE), sinusoidal নয়।

৩. Google Translate

Neural Machine Translation-এ Multi-Head Attention ব্যবহার করে source language-এর বিভিন্ন দিক (grammar, vocabulary, context) একসাথে বোঝা যায়।

৪. AlphaFold 2 (Protein Structure Prediction)

DeepMind-এর AlphaFold 2 প্রোটিনের amino acid sequence-এ Multi-Head Attention ব্যবহার করে দূরবর্তী amino acid-এর মধ্যে spatial relationship খুঁজে বের করে।

৫. Vision Transformer (ViT) — Computer Vision

Image-কে 16×16 patch-এ ভাগ করে, প্রতিটি patch একটি "token" হিসেবে। Positional Encoding patch-এর অবস্থান জানান দেয়। Google, Meta-তে image classification-এ ব্যবহৃত।

৭. Pros & Cons

সুবিধা ✓	অসুবিধা ✗
একসাথে বহু ধরনের সম্পর্ক ধরতে পারে	Computational complexity: $O(n^2 \cdot d)$ — দীর্ঘ sequence-এ ব্যয়বহুল
Parallel computation — GPU-তে দ্রুত	Memory usage অনেক বেশি (Large sequence-এ)
Single attention-এর চেয়ে অনেক সমৃদ্ধ representation	Hyperparameter tuning কঠিন (num_heads, d_model)
Theoretical ভিত্তি শক্তিশালী	Interpretability কমে যায় (অনেক head)
Sinusoidal PE-তে কোনো extra parameter নেই	Sinusoidal PE খুব দীর্ঘ sequence-এ কার্যকারিতা হারাতে পারে
বিভিন্ন domain-এ (text, image, protein) কাজ করে	Implementation জটিল — debugging কঠিন
Relative position বোঝার ক্ষমতা (PE-র কারণে)	Fixed PE নতুন ধরনের sequence structure handle করতে পারে না

৮. ⚠ Common Mistakes & Gotchas

ভুল ১: `d_model % num_heads != 0`

সবচেয়ে সাধারণ ভুল। `d_model=512`, `num_heads=7` দিলে ভাগ হয় না।

```
# ✗ ভুল
d_model = 512; num_heads = 7    # 512 / 7 = 73.14... ✗

# ✓ সঠিক
d_model = 512; num_heads = 8    # 512 / 8 = 64 ✓
```

ভুল ২: Scaling ভুলে যাওয়া

`sqrt(d_k)` দিয়ে না ভাগ করলে softmax saturate হয় এবং gradient vanish করে।

```
# X ভুল (scale নেই)
scores = torch.matmul(Q, K.transpose(-2, -1))

# ✓ সঠিক
scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
```

ভুল ৩: PE-কে Training করার চেষ্টা

Sinusoidal PE `register_buffer` দিয়ে save করতে হবে, `nn.Parameter` নয়।

```
# X ভুল – PE train হবে না, কিন্তু parameter হিসেবে ধরা হবে
self.pe = nn.Parameter(pe)

# ✓ সঠিক – PE fixed, trainable নয়
self.register_buffer('pe', pe)
```

ভুল ৪: Positional Encoding যোগ না করা

Token embedding তৈরির পরেই PE যোগ করতে হবে, Attention-এর পরে নয়।

```
# ✓ সঠিক ক্রম
x = token_embedding(tokens)      # Step 1: Word Embedding
x = positional_encoding(x)       # Step 2: PE যোগ করো
x = multi_head_attention(x)      # Step 3: তারপর Attention
```

ভুল ৫: Attention Mask ভুলে যাওয়া

Decoder-এ future token দেখা রোধ করতে **causal mask** লাগে।

```
# Causal mask তৈরি (future token দেখা যাবে না)
mask = torch.tril(torch.ones(seq_len, seq_len))
# 1 = দেখতে পারবে, 0 = দেখতে পারবে না
```

ভুল ৬: Large num_heads ধরে নেওয়া যে সবসময় ভালো

বেশি head মানেই ভালো নয়। অনেক small head অর্থহীন pattern শিখতে পারে। সবসময় `num_heads=8` বা `16` start হিসেবে ভালো।

৯. 📌 Related Topics

আগে যা জানা দরকার (Prerequisites):

- **Self-Attention (Q, K, V):** Multi-Head Attention-এর মূল building block → নোট পড়ুন
- **Word Embedding:** শব্দকে vector-এ পরিণত করা
- **Scaled Dot-Product Attention:** $Q \cdot K^T / \sqrt{d_k}$ formula
- **Softmax:** Probability distribution তৈরি
- **Matrix Multiplication:** Linear Algebra basics

পরে কী শেখা উচিত (Next Steps):

- **The Full Transformer Architecture:** Encoder-Decoder, Add & Norm, Feed Forward → পর্ব ২৫
- **BERT:** Bidirectional Encoder — Pre-training with Masked Language Model
- **GPT Architecture:** Decoder-only Transformer, Autoregressive language model
- **Sparse Attention:** Long sequence-এর জন্য $O(n^2)$ সমস্যার সমাধান (Longformer, BigBird)
- **Rotary Positional Encoding (RoPE):** LLaMA, GPT-Neo-তে ব্যবহৃত আধুনিক PE
- **ALiBi (Attention with Linear Biases):** Fixed slope দিয়ে position encoding

১০. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল:

Multi-Head Attention:

📹 "একাধিক ক্যামেরা, একটি পরিচালক"

- প্রতিটি Head = একটি ক্যামেরা (আলাদা দৃষ্টিভঙ্গি)

- Concatenation + W^O = পরিচালক সব footage জোড়া লাগাচ্ছেন

- BERT=12 heads, GPT-3=96 heads — বেশি head = বেশি ক্যামেরা

Positional Encoding:

📍 "GPS স্থানাঙ্ক কিন্তু শব্দের জন্য"

- Sin = X-coordinate (অদ্ভুত ওঠানামা করে)

- Cos = Y-coordinate (ধীরে ধীরে পরিবর্তন হয়)

- একসাথে = প্রতিটি শব্দের unique "GPS location"

Formula মনে রাখা:

```
MHA = Concat(head_1...head_h) × W_O
PE_even = sin(pos / 10000^(2i/d))
PE_odd  = cos(pos / 10000^(2i/d))
```

Number trick:

- BERT-Base: **12** heads × **12** layers (মনে রাখো "১২-১২")
- GPT-3: **96** heads (৯৬ = ৮ × ১২)
- $d_k = d_{\text{model}} / \text{num_heads}$ (সবসময় ভাগ করে নাও)

🌟 এক লাইনে সারসংক্ষেপ:

Multi-Head Attention হলো "একই সাথে ৮টি আলাদা চশমায় বাক্য পড়া" — প্রতিটি চশমা ভিন্ন সম্পর্ক দেখে; আর **Positional Encoding** হলো "প্রতিটি শব্দে পৃষ্ঠা নম্বর লিখে দেওয়া" — যাতে Transformer মনে রাখে কোন শব্দ কোথায় ছিল।

📅 তৈরিৰ তাৰিখ: ২০২৬-০৪-১১ | পূৰ্ববৰ্তী: [Self-Attention QKV](#) | পৰবৰ্তী: [Full Transformer Architecture \(২৫\)](#)

📚 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

🖨️ AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes